

Question 01

```
import matplotlib.pyplot as plt
import cv2
import numpy as np

image = cv2.imread('/content/horse.jpeg', 0)

# Get image dimensions
m, n = image.shape

# Create a 5 x 5 matrix
mask = np.ones([5, 5], dtype=int)

# Divide each value by 25 -> makes average (since 5x5 = 25 elements)
mask = mask / 25

# Create a new image with same size filled with zeros
new_image = np.zeros([m, n])

# Apply filter
for i in range(2, m-2):
    for j in range(2, n-2):
        # Multiply each neighboring filter with mask value
        temp = (
            image[i-2, j-2] * mask[0, 0] +
            image[i-2, j-1] * mask[0, 1] +
            image[i-2, j] * mask[0, 2] +
            image[i-2, j+1] * mask[0, 3] +
            image[i-2, j+2] * mask[0, 4] +

            image[i-1, j-2] * mask[1, 0] +
            image[i-1, j-1] * mask[1, 1] +
            image[i-1, j] * mask[1, 2] +
            image[i-1, j+1] * mask[1, 3] +
            image[i-1, j+2] * mask[1, 4] +

            image[i, j-2] * mask[2, 0] +
            image[i, j-1] * mask[2, 1] +
            image[i, j] * mask[2, 2] +
            image[i, j+1] * mask[2, 3] +
            image[i, j+2] * mask[2, 4] +

            image[i+1, j-2] * mask[3, 0] +
            image[i+1, j-1] * mask[3, 1] +
            image[i+1, j] * mask[3, 2] +
            image[i+1, j+1] * mask[3, 3] +
            image[i+1, j+2] * mask[3, 4] +

            image[i+2, j-2] * mask[4, 0] +
            image[i+2, j-1] * mask[4, 1] +
            image[i+2, j] * mask[4, 2] +
            image[i+2, j+1] * mask[4, 3] +
            image[i+2, j+2] * mask[4, 4]
        )

        new_image[i, j] = temp

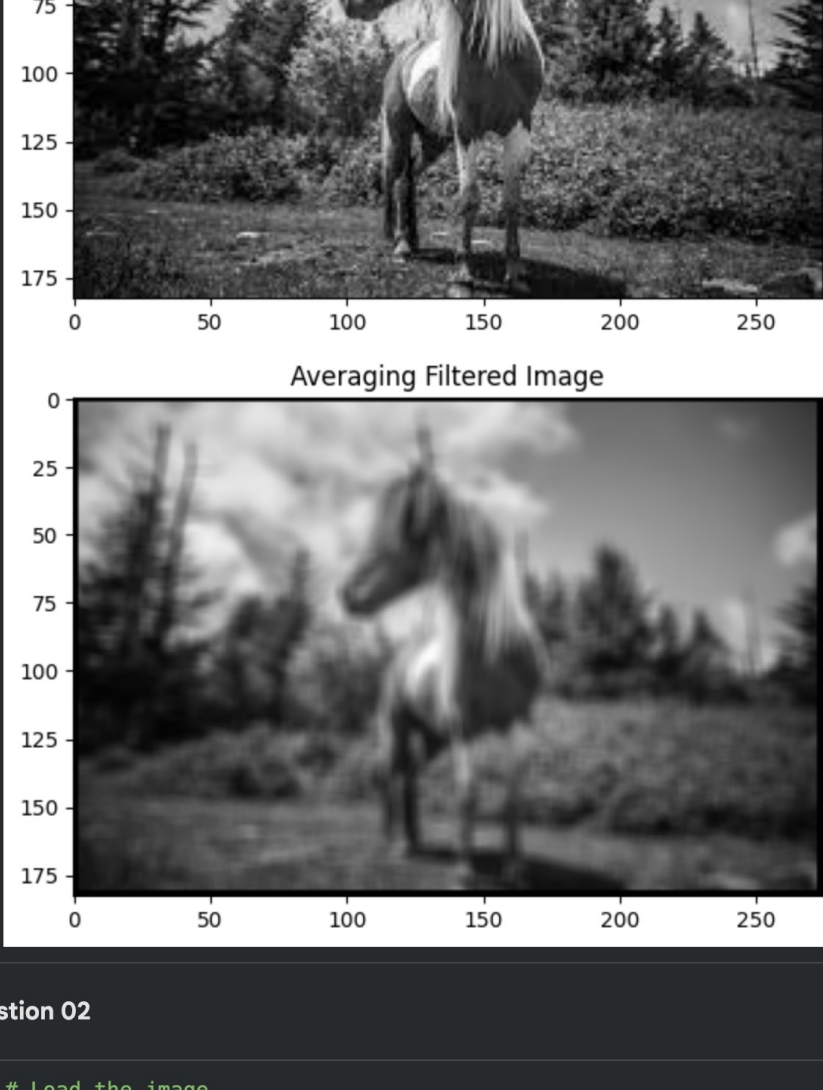
# Convert values to unsigned 8-bit integers (0-255)
new_image = new_image.astype(np.uint8)

# Save the output image
cv2.imwrite('Averaging Filtered Image.tif', new_image)
```

True

```
# Display the original image
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.show()
```

```
# Display the filtered image
plt.imshow(new_image, cmap='gray')
plt.title('Averaging Filtered Image')
plt.show()
```



Question 02

```
# Load the image
image = cv2.imread('/content/parrot.jpeg', cv2.IMREAD_COLOR)

# Convert from BGR to RGB for correct color display
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Apply Gaussian filter
kernel_size = (31, 31)
sigma_value = 0 # to define the blur strength

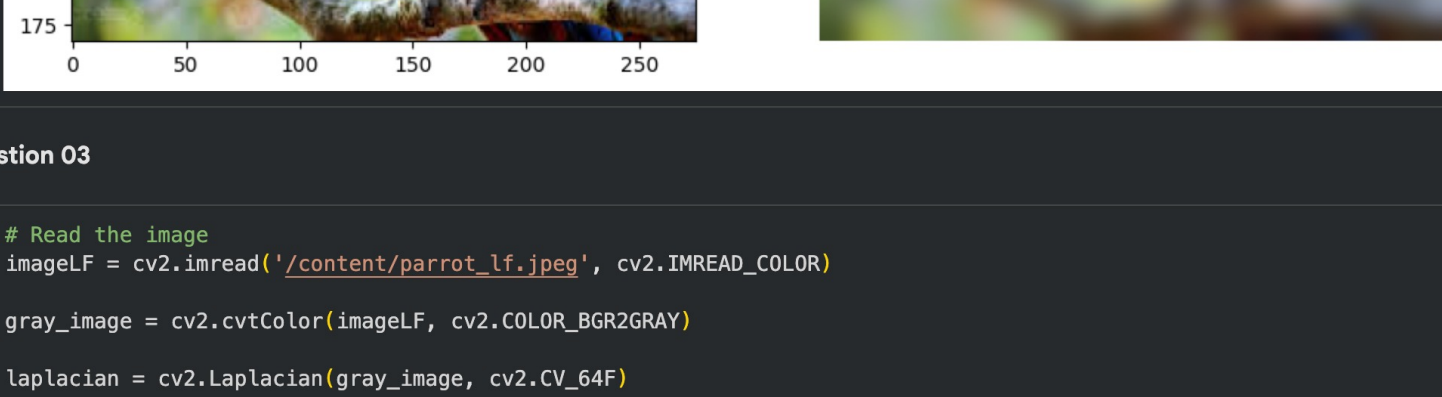
guassain_blur = cv2.GaussianBlur(image_rgb, kernel_size, sigma_value)

# Create a figure with two sub plots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Plot the original image
axes[0].imshow(image_rgb)
axes[0].set_title('Original Image')
plt.axis('off')

# Plot the blurred image
axes[1].imshow(guassain_blur)
axes[1].set_title('Blurred Image')
plt.axis('off')

# Show the plot
plt.show()
```



Question 03

```
# Read the image
imageLF = cv2.imread('/content/parrot_lf.jpeg', cv2.IMREAD_COLOR)

gray_image = cv2.cvtColor(imageLF, cv2.COLOR_BGR2GRAY)

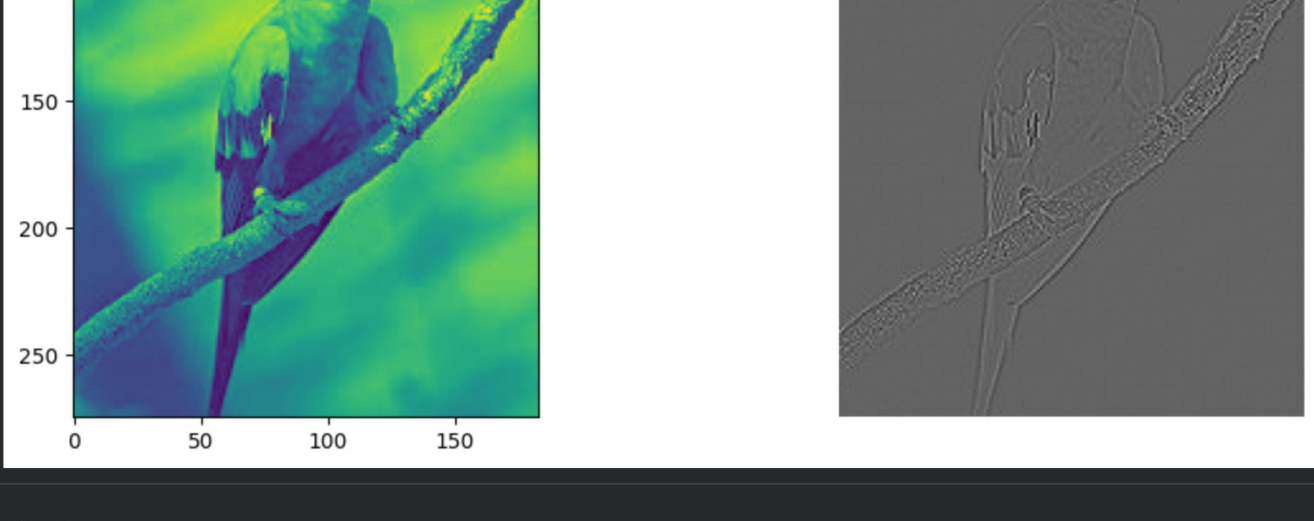
laplacian = cv2.Laplacian(gray_image, cv2.CV_64F)

# Create figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Plot the original image
axes[0].imshow(gray_image)
axes[0].set_title('Original Image Gray')
plt.axis('off')

# Plot the Laplacian image
axes[1].imshow(laplacian, cmap='gray')
axes[1].set_title('Laplacian Applied Image')
plt.axis('off')

# Show the plot
plt.show()
```



Question 04

```
Image0S = cv2.imread('/content/noisy_cameraman.jpeg', 0)

# Get image dimensions
m, n = Image0S.shape

# Create a new image with same size filled with zeros
median_filtered_image = np.zeros([m, n])

# Apply 3x3 median filter
for i in range(1, m-1):
    for j in range(1, n-1):
        # Select the 3x3 area around the current pixel
        temp = [
            Image0S[i-1, j-1],
            Image0S[i-1, j],
            Image0S[i-1, j+1],

            Image0S[i, j-1],
            Image0S[i, j],
            Image0S[i, j+1],

            Image0S[i+1, j-1],
            Image0S[i+1, j],
            Image0S[i+1, j+1]
        ]

        # Sort the values
        temp = sorted(temp)

        # Take the middle value
        median_filtered_image[i, j] = temp[4]

# Convert to uint8 format
median_filtered_image = median_filtered_image.astype(np.uint8)

# Save the filtered image
cv2.imwrite('Median Filtered Image.tif', median_filtered_image)
```

True

```
# Create figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Display original grayscale image
axes[0].imshow(Image0S, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')
```

```
# Display median filtered image
axes[1].imshow(median_filtered_image, cmap='gray')
axes[1].set_title('Median Filtered Image')
axes[1].axis('off')
```

```
# Show the plot
plt.show()
```



```
Image0S = cv2.imread('/content/noisy_circuitboard.png', 0)

# Get image dimensions
m, n = Image0S.shape

# Create a new image with same size filled with zeros
median_filtered_image = np.zeros([m, n])

# Apply 3x3 median filter
for i in range(1, m-1):
    for j in range(1, n-1):
        # Select the 3x3 area around the current pixel
        temp = [
            Image0S[i-1, j-1],
            Image0S[i-1, j],
            Image0S[i-1, j+1],

            Image0S[i, j-1],
            Image0S[i, j],
            Image0S[i, j+1],

            Image0S[i+1, j-1],
            Image0S[i+1, j],
            Image0S[i+1, j+1]
        ]

        # Sort the values
        temp = sorted(temp)

        # Take the middle value
        median_filtered_image[i, j] = temp[4]

# Convert to uint8 format
median_filtered_image = median_filtered_image.astype(np.uint8)

# Save the filtered image
cv2.imwrite('Median Filtered Image.tif', median_filtered_image)
```

True

```
# Create figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Display original grayscale image
axes[0].imshow(Image0S, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')
```

```
# Display median filtered image
axes[1].imshow(median_filtered_image, cmap='gray')
axes[1].set_title('Median Filtered Image')
axes[1].axis('off')
```

```
# Show the plot
plt.show()
```

